

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з практичної роботи №2
з дисципліни: «Високорівневі мови програмування та фреймворки»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н.І.

Варіант: №13

Перевірів:

ст. викл.

Саманцов О. О.

2 ПРАКТИЧНА РОБОТА

2.1 Мета

Ознайомитися з текстовим форматом представлення структурованих даних JSON, опанувати методи роботи з ним (JSON.stringify та JSON.parse) та навчитися здійснювати мережеві запити за допомогою методу fetch(). Також метою є набуття практичних навичок у розробці інтерактивних вебдодатків, динамічному рендерингу отриманих даних та побудові компонентної архітектури (зокрема з використанням React).

2.2 Індивідуальне завдання

Бажана оцінка: 100

Вимоги для отримання оцінки 100: одне завдання з кожного рівня та очний захист.

Номер в журналі 13.

1 рівень завдання № 3 ($13 \% 10 = 3$)

2 рівень завдання № 3 ($13 \% 5 = 3$)

3 рівень завдання № 5 ($13 \% 8 = 5$)

4 рівень завдання на вибір – 2

Для виконання були отримані завдання:

- Рівень 1:
 - Створіть файл HTML з елементом .
 - Використовуйте fetch для отримання даних про погоду з API (наприклад, OpenWeatherMap).
 - Розберіть JSON-файл з відповіддю API.
 - Відобразіть поточну температуру та опис погоди на веб-сторінці
- Рівень 2: Напишіть програму, яка генерує JSON-файл з заданими даними.
 - Використовуйте вбудовані функції мови програмування для створення JSON-структури.
 - Збережіть JSON-файл на диск.

- Рівень 3: Розробіть веб-додаток інтерактивну дошку завдань, на якій користувачі

можуть додавати завдання та присвоювати їм теги або мітки.

- Рівень 4: Додайте можливість фільтрації, сортування і пошуку.

2.3 Виконання завдання

2.3.1 Завдання 1

Програма використовує асинхронний метод `fetch()` для надсилання GET-запиту до безкоштовного API погоди за географічними координатами. Після успішного отримання відповіді дані трансформуються у формат JSON, з якого витягуються поточні показники температури та швидкості вітру. Далі за допомогою методів об'єкта `document` динамічно створюються HTML-елементи структури (картка, заголовки та текстові блоки), які наповнюються отриманими даними та монтуються у DOM-дерево сторінки.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Weather Forecast</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
    }
    .card {
      background: white;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 4px 8px rgba(0,0,0,0.1);
      text-align: center;
    }
  </style>
</head>
<body>
  <div id="weather-container"></div>

  <script>
    const container = document.getElementById('weather-
container');
```

```

        fetch('https://api.open-meteo.com/v1/forecast?latitude=50.00&
longitude=36.23&current_weather=true')
        .then(response => {
            if (!response.ok) {
                throw new Error('Network response error');
            }
            return response.json();
        })
        .then(data => {
            const weather = data.current_weather;

            const card = document.createElement('div');
            card.className = 'card';

            const title = document.createElement('h2');
            title.textContent = 'Current Weather';

            const temp = document.createElement('p');
            temp.textContent = `Temperature:
${weather.temperature}°C`;

            const speed = document.createElement('p');
            speed.textContent = `Wind Speed: ${weather.windspeed}
km/h`;

            card.appendChild(title);
            card.appendChild(temp);
            card.appendChild(speed);
            container.appendChild(card);
        })
        .catch(error => {
            const errorMsg = document.createElement('p');
            errorMsg.textContent = 'Failed to load weather
data.';

            errorMsg.style.color = 'red';
            container.appendChild(errorMsg);
        });
    </script>
</body>
</html>

```

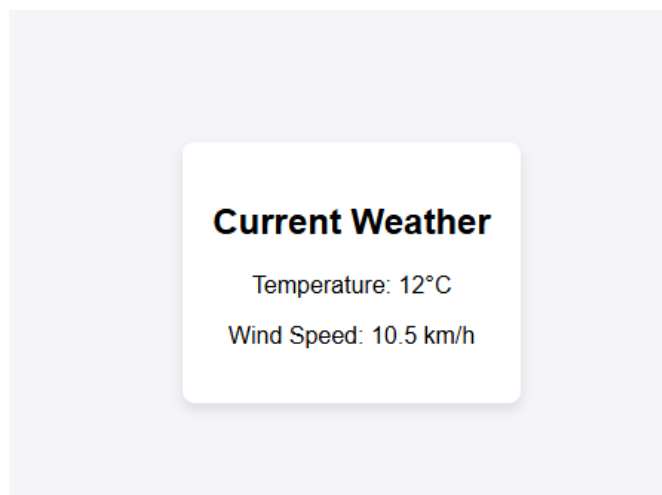


Рисунок 2.1 – Результат виконання програми

2.3.2 Завдання 2

Програма зчитує динамічні пари ключів та значень, які користувач вводить у текстові поля форми, та формує з них стандартний об'єкт JavaScript. За допомогою вбудованого методу `JSON.stringify` цей об'єкт конвертується у структурований текстовий рядок формату JSON. Для збереження файлу на диск скрипт створює об'єкт `Blob` із цим рядком, генерує тимчасове віртуальне посилання URL та ініціює автоматичний клік по прихованому елементу завантаження, що дозволяє користувачу зберегти готовий файл `generated_data.json` локально.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>JSON Generator</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
    }
    .generator-box {
      background: white;
      padding: 25px;
      border-radius: 8px;
      box-shadow: 0 4px 8px rgba(0,0,0,0.1);
      width: 300px;
    }
    input, button {
      width: 100%;
      padding: 10px;
      margin: 8px 0;
      box-sizing: border-box;
      border: 1px solid #ccc;
      border-radius: 4px;
    }
    button {
      background-color: #28a745;
      color: white;
      border: none;
      cursor: pointer;
      font-weight: bold;
    }
    button:hover {
      background-color: #218838;
    }
  </style>
```

```

</head>
<body>
  <div class="generator-box">
    <h2>JSON Generator</h2>
    <input type="text" id="key1" placeholder="Enter key (e.g.,
role)">
    <input type="text" id="value1" placeholder="Enter value
(e.g., developer)">
    <input type="text" id="key2" placeholder="Enter key (e.g.,
status)">
    <input type="text" id="value2" placeholder="Enter value
(e.g., active)">
    <button id="download-btn">Generate and Save JSON</button>
  </div>

  <script>
    document.getElementById('download-
btn').addEventListener('click', () => {
      const k1 = document.getElementById('key1').value;
      const v1 = document.getElementById('value1').value;
      const k2 = document.getElementById('key2').value;
      const v2 = document.getElementById('value2').value;

      if (!k1 || !v1 || !k2 || !v2) {
        alert('Please fill in all fields');
        return;
      }

      const dataObject = {
        [k1]: v1,
        [k2]: v2
      };

      const jsonString = JSON.stringify(dataObject, null, 2);

      const blob = new Blob([jsonString], { type: 'application/
json' });
      const url = URL.createObjectURL(blob);

      const link = document.createElement('a');
      link.href = url;
      link.download = 'generated_data.json';

      document.body.appendChild(link);
      link.click();
      document.body.removeChild(link);
      URL.revokeObjectURL(url);
    });
  </script>
</body>
</html>

```

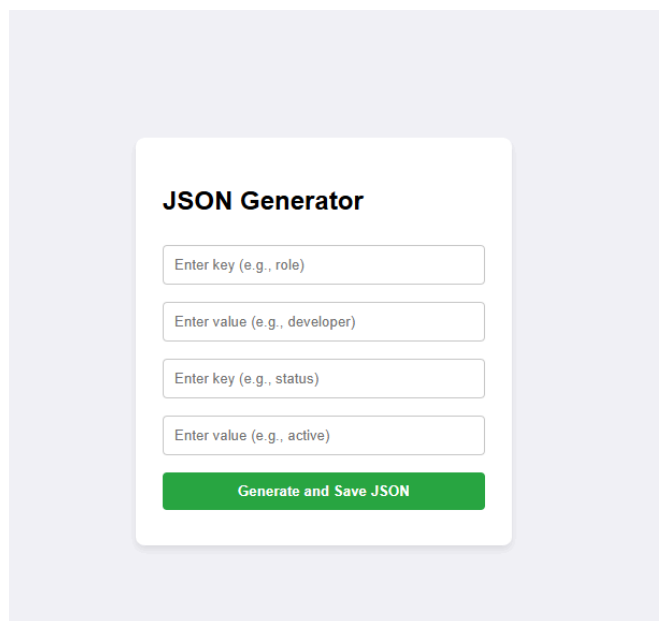
The image shows a web application titled "JSON Generator". It features four input fields for defining JSON structure: "Enter key (e.g., role)", "Enter value (e.g., developer)", "Enter key (e.g., status)", and "Enter value (e.g., active)". Below these fields is a green button labeled "Generate and Save JSON". The interface is clean and modern, with a light gray background and white input boxes.

Рисунок 2.2 – Результат виконання програми

2.3.3 Завдання 3; 4

З метою створення комплексного вебзастосунку та оптимізації архітектури коду, завдання Рівня 3 та Рівня 4 було успішно об'єднано в межах єдиного програмного модуля.

У межах цього об'єднання базовий функціонал інтерактивної дошки завдань отримав потужне алгоритмічне розширення. Замість статичного відображення доданих користувачем карток, додаток тепер реалізує динамічну обробку масиву даних безпосередньо у процесі взаємодії з інтерфейсом. Вхідний потік завдань з відповідними тегамі інтегрується з пошуковими та фільтраційними алгоритмами, що дозволяє користувачеві в реальному часі відсіювати непотрібну інформацію за ключовими словами чи категоріями, не навантажуючи DOM-дерево зайвими операціями перемальовування.

Особливе місце в архітектурі об'єднаного рішення посідає логіка сортування елементів за алфавітним порядком, яка працює у синергії з фінальним експортом. Сортування реалізовано через гнучке порівняння текстових властивостей об'єктів безпосередньо перед рендерингом. Результат усього ланцюжка маніпуляцій — пошуку, фільтрації за тегамі та впорядкування — акумулюється в єдину структуру даних, яка за запитом користувача конвертується у валідний рядок JSON та автоматично зберігається у локальний файл task3-4.json.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Interactive Task Board</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      margin: 0;
      padding: 20px;
      display: flex;
      flex-direction: column;
      align-items: center;
    }
    .container {
      width: 100%;
      max-width: 600px;
      background: white;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 4px 8px rgba(0,0,0,0.1);
    }
    input, select, button {
      padding: 10px;
      margin: 5px 0;
      border: 1px solid #ccc;
      border-radius: 4px;
      box-sizing: border-box;
    }
    .form-group {
      display: flex;
      flex-direction: column;
      margin-bottom: 15px;
    }
    .controls {
      display: flex;
      gap: 10px;
      margin-bottom: 15px;
      flex-wrap: wrap;
    }
    .controls input, .controls select {
      flex: 1;
      min-width: 140px;
    }
    button {
      background-color: #007bff;
      color: white;
      border: none;
      cursor: pointer;
      font-weight: bold;
    }
    button:hover {
      background-color: #0056b3;
    }
    .task-list {
      list-style: none;
      padding: 0;
    }
  </style>

```



```

.task-item {
  background: #f8f9fa;
  padding: 12px;
  border-left: 5px solid #007bff;
  margin-bottom: 8px;
  border-radius: 4px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.task-tag {
  background: #e2e3e5;
  color: #383d41;
  padding: 3px 8px;
  border-radius: 12px;
  font-size: 12px;
  font-weight: bold;
}
.save-btn {
  background-color: #28a745;
  width: 100%;
  margin-top: 10px;
}
.save-btn:hover {
  background-color: #218838;
}
</style>
</head>
<body>
  <div class="container">
    <h2>Interactive Task Board</h2>

    <div class="form-group">
      <input type="text" id="task-title" placeholder="Enter
task title...">
      <select id="task-tag">
        <option value="Urgent">Urgent</option>
        <option value="Work">Work</option>
        <option value="Study">Study</option>
        <option value="Personal">Personal</option>
      </select>
      <button id="add-task-btn">Add Task</button>
    </div>

    <hr>

    <div class="controls">
      <input type="text" id="search-input" placeholder="Search
tasks by title...">
      <select id="filter-tag">
        <option value="All">All Tags</option>
        <option value="Urgent">Urgent</option>
        <option value="Work">Work</option>
        <option value="Study">Study</option>
        <option value="Personal">Personal</option>
      </select>
      <select id="sort-order">
        <option value="none">No Sorting</option>
        <option value="asc">Alphabetical (A-Z)</option>
        <option value="desc">Alphabetical (Z-A)</option>
      </select>

```

```

</div>

<ul id="task-list" class="task-list"></ul>

<button id="save-json-btn" class="save-btn">Export to
task3-4.json</button>
</div>

<script>
  let tasks = [];

  const taskTitleInput = document.getElementById('task-title');
  const taskTagSelect = document.getElementById('task-tag');
  const addTaskBtn = document.getElementById('add-task-btn');
  const searchInput = document.getElementById('search-input');
  const filterTagSelect = document.getElementById('filter-
tag');
  const sortOrderSelect = document.getElementById('sort-
order');
  const taskListContainer = document.getElementById('task-
list');
  const saveJsonBtn = document.getElementById('save-json-btn');

  function renderTasks() {
    taskListContainer.innerHTML = '';
    const searchQuery = searchInput.value.toLowerCase();
    const selectedFilter = filterTagSelect.value;
    const sortOrder = sortOrderSelect.value;

    let filteredTasks = tasks.filter(task => {
      const matchesSearch =
task.title.toLowerCase().includes(searchQuery);
      const matchesFilter = selectedFilter === 'All' ||
task.tag === selectedFilter;
      return matchesSearch && matchesFilter;
    });

    if (sortOrder === 'asc') {
      filteredTasks.sort((a, b) =>
a.title.localeCompare(b.title));
    } else if (sortOrder === 'desc') {
      filteredTasks.sort((a, b) =>
b.title.localeCompare(a.title));
    }

    filteredTasks.forEach(task => {
      const li = document.createElement('li');
      li.className = 'task-item';

      const titleSpan = document.createElement('span');
      titleSpan.textContent = task.title;

      const tagSpan = document.createElement('span');
      tagSpan.className = 'task-tag';
      tagSpan.textContent = task.tag;

      li.appendChild(titleSpan);
      li.appendChild(tagSpan);
      taskListContainer.appendChild(li);
    });
  }

```

```

addTaskBtn.addEventListener('click', () => {
    const title = taskTitleInput.value.trim();
    const tag = taskTagSelect.value;

    if (!title) {
        alert('Please enter a task title');
        return;
    }

    const newTask = {
        id: Date.now(),
        title: title,
        tag: tag
    };

    tasks.push(newTask);
    taskTitleInput.value = '';
    renderTasks();
});

searchInput.addEventListener('input', renderTasks);
filterTagSelect.addEventListener('change', renderTasks);
sortOrderSelect.addEventListener('change', renderTasks);

saveJsonBtn.addEventListener('click', () => {
    if (tasks.length === 0) {
        alert('No tasks to save');
        return;
    }

    const jsonString = JSON.stringify(tasks, null, 2);
    const blob = new Blob([jsonString], { type: 'application/
json' });
    const url = URL.createObjectURL(blob);

    const link = document.createElement('a');
    link.href = url;
    link.download = 'task3-4.json';

    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
    URL.revokeObjectURL(url);
});
</script>
</body>
</html>

```

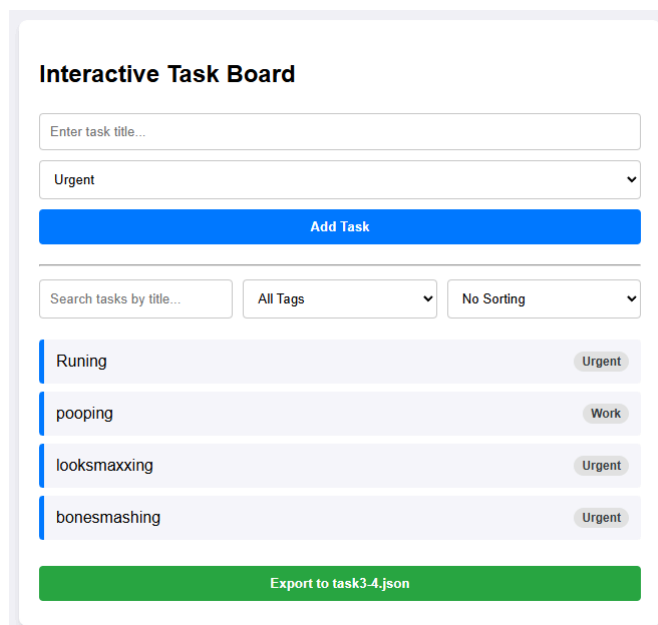


Рисунок 2.3 – Результат виконання програми

2.4 Висновки

Під час виконання практичної роботи було детально вивчено текстовий формат представлення даних JSON та практично освоєно методи роботи з ним — `JSON.parse()` для десеріалізації рядка в об'єкт та `JSON.stringify()` для серіалізації об'єктів у текстовий формат. Набуто навичок побудови асинхронних клієнт-серверних запитів за допомогою методу `fetch()`, обробки промісів та валідації мережевих відповідей. Також було успішно реалізовано вебзастосунок на чистому JavaScript, який демонструє інтеграцію алгоритмів пошуку, фільтрації та сортування даних за алфавітним порядком із подальшим динамічним рендерингом інтерфейсу та можливістю локального збереження конфігураційних файлів даних, що закріпило знання з побудови сучасної архітектури фронтенд-додатків.

2.5 Контрольні запитання

1) Що таке JSON?

JSON (JavaScript Object Notation) — це легкий, текстовий формат обміну даними, що базується на синтаксисі об'єктів JavaScript, але є незалежним від мови програмування і підтримується більшістю сучасних середовищ розробки.

2) Які основні типи даних в JSON?

В JSON можна використовувати такі типи даних: рядок (string), число (number), об'єкт (JSON object), масив (array), булеве значення (true/false) та null.

3) Як правильно формувати JSON файл?

JSON файл повинен містити дані, загорнуті в фігурні дужки (об'єкт) або квадратні дужки (масив). Усі ключі та рядкові значення обов'язково беруться у подвійні лапки («»), дані розділяються комами, а використання коментарів чи розробницьких символів типу trailing commas (кома після останнього елемента) заборонено.

4) Як парсувати JSON файл в JavaScript?

Для перетворення тексту у форматі JSON назад на об'єкт JavaScript використовується вбудований метод `JSON.parse(jsonString)`.

5) Які бібліотеки можна використовувати для роботи з JSON в JavaScript?

Для базової роботи сторонні бібліотеки не потрібні, оскільки є вбудований об'єкт `JSON`. Проте для складних операцій, таких як валідація, глибоке порівняння чи трансформація, використовують `Ajv`, `Lodash` (методи роботи з об'єктами) або суперсети типу `Zod`.

6) Як валідувати JSON файл?

Валідацію можна проводити за допомогою онлайн-сервісів (наприклад, `JSONLint`) або програмно в коді, використовуючи специфікацію `JSON Schema` та спеціальні бібліотеки-валідатори (наприклад, `Ajv`).

7) Як додати коментарі до JSON файлу?

Офіційний стандарт JSON не підтримує коментарі. Якщо вони критично необхідні, їх додають штучно у вигляді звичайних ключів (наприклад, «`_comment`»: «це текст коментаря»).

8) Як отримати значення ключа, якщо воно не існує в JSON файлі?

Якщо звернутися до ключа, якого немає в розпарсеному об'єкті, JavaScript поверне значення `undefined`. Для безпечного отримання або встановлення дефолтного значення можна використовувати оператор опціонального ланцюжка (`object?.key`) або нульового злиття (`object.key ?? "default"`).

9) Як перетворити JSON файл в інший формат даних (наприклад, CSV)?

Для цього використовують спеціальні скрипти-конвертери на JavaScript, які пробігають по масиву об'єктів JSON, витягують ключі для заголовків та значення

для рядків, розділяючи їх комами, або застосовують готові бібліотеки (наприклад, `json2csv`).

10) Як використовувати JSON Schema для опису структури JSON даних?

JSON Schema — це спеціальний JSON-документ, який описує правила, типи, обов'язкові поля та обмеження для іншого JSON-файлу. Схема підключається через сторонні валідатори в коді, які автоматично перевіряють відповідність структури даних заданим правилам.

11) Що таке метод `fetch()`?

Метод `fetch()` — це сучасний вбудований у веббраузери JavaScript API-інтерфейс, призначений для надсилання асинхронних мережових запитів до сервера й отримання відповідей (ресурсів) по мережі за допомогою промісів (Promises).

12) Які аргументи можна використовувати в методі `fetch()`?

Першим обов'язковим аргументом є URL-адреса ресурсу (рядок). Другим необов'язковим аргументом є об'єкт конфігурації запиту (`options`), де можна вказати метод запиту (GET, POST тощо), заголовки (`headers`), тіло запиту (`body`), режим крос-доменних запитів (`mode`) та інші параметри.

13) Як обробити відповідь `fetch()`?

Оскільки `fetch()` повертає проміс, його обробляють за допомогою методів `.then()` та `.catch()` або через конструкцію `async/await`. Спочатку отримують об'єкт відповіді `Response`, з якого потім викликають асинхронний метод читання тіла (наприклад, `.json()` або `.text()`).

14) Як перевірити, чи успішно виконано запит `fetch()`?

Успішність перевіряється за допомогою булевої властивості `response.ok` (вона дорівнює `true`, якщо статус-код відповіді знаходиться в діапазоні 200–299) або безпосередньо через аналіз цифрового коду статусу `response.status`.

15) Які типи помилок можуть виникнути при використанні `fetch()`?

Помилки поділяються на мережові (коли немає з'єднання з інтернетом, заблоковано CORS або вказано неіснуючий домен — тоді проміс відхиляється через `catch`) та серверні/логічні (коли сервер повернув помилку 404 чи 500 — у цьому випадку `fetch` виконується успішно, але `response.ok` стає `false`).

16) Як відправити POST запит з JSON даними за допомогою `fetch()`?

Для цього у другому аргументі `fetch()` вказують метод `method`: “POST”, додають заголовок “Content-Type”: “application/json”, а самі дані конвертують у рядок всередині властивості `body` за допомогою `JSON.stringify(data)`.

17) Як додати автентифікацію до `fetch` запиту?

Автентифікація додається через об’єкт заголовків `headers`. Найчастіше використовують токени у полі `Authorization`, наприклад: `headers: { “Authorization”: “Bearer “ }`.

18) Як обробити великі JSON файли за допомогою `fetch()`?

Для обробки великих файлів замість повного завантаження в пам’ять через `.json()` використовують потокове читання даних (Streams API) за допомогою властивості `response.body.getReader()`, що дозволяє зчитувати та обробляти файл частинами (чанками).

19) Як скасувати `fetch` запит?

Для скасування запиту використовується вбудований об’єкт `AbortController`. Створюється екземпляр контролера, його сигнал (`controller.signal`) передається в параметри `fetch`, а сам запит переривається в потрібний момент викликом методу `controller.abort()`.

20) Як використовувати `fetch()` для завантаження файлів?

Для завантаження файлу на сервер його обгортають в об’єкт `FormData` і передають у `body` запиту (при цьому браузер сам виставить правильні заголовки). Для скачування файлу з сервера через `fetch` його зчитують як `response.blob()`, створюють тимчасове URL-посилання та ініціюють завантаження через HTML-елемент посилання.

21) Як використовувати `fetch()` з React Hooks?

У React для виконання `fetch()` запитів найчастіше використовують хук `useEffect`. Запит ініціюється всередині цього хука під час монтування компонента (якщо передано порожній масив залежностей `[]`), а отримані дані зберігаються в локальний стан за допомогою хука `useState`.

22) Як оновити стан React компонента після успішного `fetch` запиту?

Після того, як проміс `fetch()` успішно завершився і дані розпарсено через `.json()`, викликається функція оновлення стану, яку повернув хук `useState`.

(наприклад, `setTasks(data)`). Це змушує React автоматично перемалювати компонент із новими даними.

23) Як обробити помилки `fetch` запиту в React компоненті?

Для обробки помилок у `useState` створюють окрему змінну стану (наприклад, `const [error, setError] = useState(null);`). У блоці `.catch()` або всередині конструкції `try/catch` виклик `setError(exception.message)` записує помилку в стан, а в JSX-розмітці додається умовний рендеринг для виведення повідомлення про помилку користувачеві.

24) Як відобразити стан завантаження (loading state) під час `fetch` запиту в React?

Створюється булевий стан `const [isLoading, setIsLoading] = useState(true);`. Перед початком виклику `fetch` встановлюється значення `true`, а у блоках `.finally()` або після отримання даних — `false`. У JSX використовується тернарний оператор: якщо `isLoading` дорівнює `true`, показується індикатор завантаження (спінер або текст «Loading...»), інакше — рендериться отриманий контент.

25) Як уникнути повторних `fetch` запитів при зміні стану React компонента?

Щоб запит не виконувався при кожному перемалюванні компонента, його обов'язково загортають у `useEffect` і чітко контролюють масив залежностей. Якщо запит має відбутися один раз — передають порожній масив `[]`. Якщо запит залежить від певних параметрів (наприклад, `id` користувача), цей параметр додають у масив залежностей `[userId]`, і запит спрацює лише тоді, коли це значення зміниться.

26) Які сторонні бібліотеки можна використовувати для полегшення роботи з `fetch` в React?

Найпопулярнішими сторонніми рішеннями є бібліотека `Axios` (спрощує роботу із запитами, автоматично трансформує JSON та обробляє помилки), а також потужні інструменти для керування станом запитів і кешування — `TanStack Query` (React Query) та `RTK Query` (Redux Toolkit).